



SHAHEED NANDKUMAR PATEL VISHWAVIDYALAYA

ANANT COLLEGE, RAIGARH (C.G.)

College Code - **829**

A Project on

“ATTENDANCE MANAGEMENT SYSTEM”

By

“JITESH KHARE”

A Project report submitted in partial fulfilment of requirement

For the degree of

BACHELOR OF COMPUTER APPLICATION

Of Shaheed Nandkumar Patel Vishwavidyalaya, Raigarh (C.G.)



ANANT COLLEGE, RAIGARH (C.G.)



PROJECT WORK EVALUATION

1. Project Title: **ATTENDANCE MANAGEMENT SYSTEM**
2. Software Base: **Python**
3. Submitted for the Course& Year: **BCA (2025-26)**
4. Individual Evaluation: **Maximum 100**

NAME	ROLL NUMBER	INDIVIDUAL EVALUATION (M.G.) (MAX.100)
JITESH KHARE		

6. College Name: **ANANT COLLEGE, RAIGARH (C.G.) 829**

External Examiner

ANANT COLLEGE, RAIGARH (C.G)
BACHELOR OF COMPUTER APPLICATION

CERTIFICATE OF APPROVAL

This is to certify that the work embodied in this Minor Project, “**ATTENDANCE MANAGEMENT SYSTEM**” has been satisfactorily completed by “**JITESH KHARE**” with Enrolment Number. It is a bona fide piece of work, carried out under my guidance in the Bachelor of Computer Application, **ANANT COLLEGE**, and RAIGARH for the partial fulfilment of the Bachelor of Computer Application degree during the **Session 2025-2026**.

Guided By
Name of Guidance
MR. RANJEET PRAJAPATI

Approved By
In charge Principal
MISS. NAMRATA JALCHHATRI
ANANT COLLEGE, RAIGARH (C.G.)

ANANT COLLEGE, RAIGARH (C.G)
BECHELOR OF COMPUTER APPLICATION

CERTIFICATE OF APPROVAL

This foregoing Minor project is hereby approved as a creditable study of a computer Application Subject carried out and presented in a manner satisfactory to warranty its acceptances as a prerequisite to the degree for which it has been submitted. It is understood that by this approval the undersigned do not necessarily endorse or approval any statements made , opinion expressed on conclusion drawn therein, but approval the thesis only for the purpose the which it has been submitted.

(Internal Examiner)

(External Examiner)

ANANT COLLEGE, RAIGARH (C.G)
BECHELOR OF COMPUTER APPLICATION

DECLARATION

I, **JITESH KHARE** ,a student of bachelor of computer application ,
ANANT COLLEGE,RAIGARH hereby declare that the work
presented in this Minor Project is outcome of my own work, is
bona fide ,correct to the best of my knowledge and this work has
been carried out taking care of Engineering Ethics. The work
presented does not infringe any patented work and has not been
submitted to any University for the award of degree or any
professional degree.

NAME OF STUDENT - **JITESH KHARE**

CLASS NAME - **BCA III YEAR**

ENROLMENT NO -

ACKNOWLEDGMENT

“TASK successful makes everyone happy, but the happiness will not be gold without glitter if we didn’t state the persons how have supported us to make it a success

Success will be crowned to people how made it a reality but the people whose constants guidance and encouragement it possible will be crowned first on the eve of success.

This acknowledgement transcends the reality of formality when we would like to express deep gratitude and respect to all those people behind the screen who guided, inspired and happened me for the compilation of our project work.

I consider my self lucky in enough to get such a good project. This project would add as an asset to my academic profile.

I would like to express my thankfulness EXAM CONTROLLER of **ANANT COLLEGE, RAIGARH MR. CHANDRAKANT JALCHHATRI**, HOD of IT and my project Guide **MR. RANJEET PRAJAPATI**, for his constant motivation and valuable help through the project work.

I also extend my thanks to my team members and family members for their co-operation during my course.

Thank you!

INDEX

S.N.	DESCRIPTION	PAGE NO
1	INTRODUCTION	8
2	TECHNOLOGY ➤ Operating System ➤ Programming Language ➤ Other Tools and Technology ➤ Database ➤ Server-Web/Application	9
3	DESCRIPTION OF PROJECT	10
4	FEATURES OF PROJECT	11
5	OBJECTIVE OF PROJECT	14
6	DOMAIN OF PROJECT	17
7	MODULES	18
8	HARDWARE AND SOFTWARE REQUIREMENT	19
9	ANALYSIS	20
10	DETAILS OF MODULES ➤ Admin ➤ Client	22
11	TECHNOLOGY INTRODUCTION ➤ Technology ➤ Front End ➤ Back End	23
12	PROJECT ARCHITECTURE	26
13	DATA MODEL ➤ DFD Level 0 ➤ DFD Level 1(Admin) ➤ DFD Level 1(Client) ➤ E-R Diagram ➤ Data Dictionary ➤ Data Dependency	28
14	FRONT END DESIGN (FORM)	30
15	TESTING AND REPORT ➤ Black Box Testing ➤ White Box Testing ➤ Smoke Testing ➤ Requirement Based Testing ➤ Partition Testing ➤ Test Process	33
16	REPORT	45
17	IMPLEMENTATION	49
18	FUTURE EMHANCEMENTS	50
19	BIBLOGRAPHY ➤ BOOKS ➤ WEBSITES	51

1. INTRODUCTION

The Attendance Management System is a desktop-based application developed to streamline the process of recording and managing daily attendance. In many educational and corporate environments, maintaining attendance manually using paper registers is a tedious, error-prone, and time-consuming task. This project addresses these challenges by providing a digital interface where student information can be stored and their daily presence or absence can be logged with a single click.

The system utilizes a Graphical User Interface (GUI) to make it accessible for users with varying levels of technical expertise. By automating the calculation of time and date stamps, the software ensures high data integrity and prevents the common "human errors" associated with manual logging. The application serves as a bridge between traditional record-keeping and modern digital management, offering a reliable way to store, retrieve, and display attendance logs in real-time.

2. TECHNOLOGY

This project is built using a combination of robust and widely-accepted technologies in the field of software development.

Operating System: Windows 10/11, macOS, or Linux (Cross-platform compatibility via Python).

Programming Language: Python 3.x. Python was chosen for its readability, vast library support, and ease of integration with database systems.

Other Tools and Technology: Tkinter: The standard Python interface to the Tk GUI toolkit. It is used to create the windows, buttons, and input fields.

Datetime Library: Used for capturing the precise system date and time when attendance is marked.

Database: SQLite3. A C-language library that implements a small, fast, self-contained, high-reliability, full-featured, SQL database engine. It is server less and requires zero configuration, making it perfect for desktop applications.

Server-Web/Application: As this is a standalone desktop application, it does not require a web server. However, it functions as a Client-Side Desktop Application where the logic and data storage reside locally on the machine.

3. DESCRIPTION OF PROJECT

The project is a lightweight, efficient Attendance Management System. Upon launching the application, the user is presented with a clean interface featuring input fields for Student ID and Name. The core logic involves two primary database tables: one for "Students" (to register new individuals) and one for "Attendance" (to log daily entries).

The application allows an administrator or teacher to register students into a local SQLite database. Once registered, their attendance can be marked as "Present" or "Absent." Every time attendance is marked, the system automatically captures the current date and time from the computer's system clock. This data is then displayed in a Tree view widget, which acts as a real-time log, showing the most recent entries at the top.

4. FEATURES OF PROJECT :

1. Automated Student Registration

The system allows the administrator to enroll new students into the database. By capturing a unique Student ID and Name, the system creates a permanent record for each individual.

* **Validation:** It ensures that no two students have the same ID using SQL Primary Key constraints.

* **User Feedback:** The system provides pop-up notifications (Success/Error) to confirm whether the registration was successful.

2. One-Click Attendance Logging

The core functionality revolves around two primary actions: Mark Present and Mark Absent

* Unlike manual registers where you have to find the correct date and column, this system allows the user to simply enter an ID and click a button.

* The system handles the "Business Logic" of checking if the student exists before allowing an entry to be made.

3. Real-Time System Time-Stamping

This is a critical feature for data integrity. The system uses Python's ``datetime`` library to fetch the exact current date and time from the computer's operating system.

- * **Accuracy:** It eliminates "back-dating" or manual entry errors.
- * **Transparency:** Every attendance record is tagged with a precise time (e.g., ``14:30:05``), providing a clear audit trail of when the student was marked.

4. Dynamic Data Visualization (Tree view)

The project features a live log table powered by the ``ttk.Treeview`` widget.

- * **Instant Updates:** As soon as attendance is marked, the table refreshes to show the new entry at the top.
- * **Organization:** Data is displayed in clean, sortable columns (ID, Name, Date, Time, Status), making it easy to read at a glance.

5. Persistent Data Storage

The system is integrated with an SQLite3 database

* **Reliability:** Unlike storing data in a text file or a list, a database ensures that even if the computer crashes or the application is closed, the data remains safe.

* **Scalability:** The database can handle thousands of entries without slowing down the application's performance.

6. Intuitive Graphical User Interface (GUI)

Built with Tkinter, the interface is designed to be "user-agnostic," meaning even someone with very basic computer knowledge can operate it.

* **Color-Coded Buttons:** Visual cues like Green for "Present" and Red for "Absent" make the workflow faster and reduce accidental clicks.* **Clean Layout:** By using frames and padding, the interface remains organized and uncluttered on screens of various size

5. OBJECTIVE OF PROJECT :

1. Automation of Manual Processes

The primary objective is to transition from a manual, paper-based attendance system to an automated digital environment. Manual systems are prone to human error, such as marking the wrong student or losing physical registers. This project aims to digitize the entire workflow—from registration to daily logging.

2. Ensuring Data Integrity and Accuracy

By using a relational database (SQLite) and automated time-stamping, the project aims to provide a "single source of truth."

Preventing Proxy/Duplicates: Through the use of unique IDs, the system ensures that each student is a distinct entity.

Precise Time-Tracking: One of the main goals is to record exactly when a student arrived, which is difficult to track accurately with a pen and paper.

3. Real-Time Data Retrieval and Reporting

In traditional systems, calculating attendance percentages or finding a specific date's record requires manual counting. A core objective of this software is to provide instant access to data.

Users should be able to see the latest logs immediately after clicking a button.

The system organizes data chronologically, allowing administrators to monitor attendance patterns in real-time.

4. Resource Efficiency and Sustainability

The project aims to reduce the overhead costs associated with administrative tasks:

Paperless Environment: By moving to a digital database, the project supports eco-friendly practices by eliminating the need for physical stationery.

Time Management: The system reduces the time spent by teachers or managers on administrative "housekeeping," allowing them to focus more on their core duties.

5. Reliability and Security

Storage of records in a local database ensures that data is persistent. Unlike spreadsheets or text files, which can be easily deleted or corrupted, the project aims to use SQLite to provide a robust structure that maintains data even after the application is closed or the system is restarted.

6. Simplification of User Interaction

A major objective is to design a tool that requires zero training. The use of a Graphical User Interface (GUI) with clear labels and color-coded buttons ensures that anyone, regardless of their technical background, can successfully manage student attendance without needing to understand the underlying SQL queries.

6. DOMAIN OF PROJECT :

Domain Element	Description
System Type	MIS (Management Information System)
User Interface	GUI (Graphical User Interface)
Backend Environment	Local Server Environment (XAMPP)
Data Architecture	Relational Database Model

7. MODULES

Component	Web-Based (Easy)	Modern/Scalable
Frontend	HTML, CSS, JavaScript	React.js or Vue.js
Backend	PHP or Python (Flask)	Node.js or Python (Django)
Database	MySQL	MongoDB or PostgreSQL

8. HARDWARE AND SOFTWARE REQEIRMENT

Component	Specification
Operating System	Windows 10 or higher
Processor	Intel i3 2.4 GHz
RAM	4 GB
Database	MySQL
Server	Apache (XAMPP Control Panel)
Web Browser	Google Chrome / Mozilla Firefox

9. ANALYSIS :

1. Requirement Analysis

This involves identifying two types of requirements:

Functional Requirements (What should the system do?):

User Login: Separate portals for Admin, Teacher/Manager, and Students/Employees.

Marking Attendance: Recording attendance via manual entry, QR code, Biometrics, or RFID.

Report Generation: Daily, weekly, and monthly reports in PDF/Excel formats.

Leave Management: Application for leave and an approval workflow.

Real-time Dashboard: An instant view of the total number of present and absent individuals.

Non-Functional Requirements (System Quality):

Security: Ensuring no data leaks or unauthorized access.

Scalability: The system should not crash as the number of users increases.

Accuracy: There should be no margin for error in attendance records.

2. System Design Analysis (Process Flow)

To understand the system structure, diagrams are used:

Data Flow Diagram (DFD): Shows how data enters the system, flows through processes, and where it is stored.

Entity-Relationship (ER) Diagram: Illustrates the connections between database tables (e.g., the relationship between Student, Class, and Attendance).

3. Feasibility Study

Before starting, it is essential to verify:

Technical Feasibility: Do we have the necessary software (Python, Java, PHP) and hardware (Biometric scanners, Servers)?

Operational Feasibility: Can the end-users (teachers/students) easily operate this new system?

Economic Feasibility: Is the project within the allocated budget?

4. Proposed Technology Stack

The analysis must specify the tools to be used:

Frontend: HTML, CSS, JavaScript (React or Angular).

Backend: Node.js, Python (Django/Flask), or PHP.

Database: MySQL, PostgreSQL, or MongoDB.

5. Software Development Life Cycle (SDLC)

You must mention the development model being followed (e.g., Waterfall or Agile). Agile is currently more popular due to its fast feedback loops.

Tip: The outcome of the analysis phase is the SRS (Software Requirement Specification) document, which serves as the project's blueprint.

10. DETAILS OF MODULES

➤ Admin

➤ Client

Feature	Admin Module	Client Module
Control	Full System Access	Limited to Own Profile
Data Entry	Add/Remove Users	View/Apply Leave
Reporting	All Users' Reports	Personal Report Only

11. TECHNOLOGY INTRODUCTION

Technology

1. The Power of Python in AMS

Python is chosen for this system because of its extensive library support and ease of integration with hardware (cameras). It allows us to process high-definition video frames in real-time to identify individuals instantly.

2. Core Technical Architecture

The system is built on a layered architecture that handles everything from light sensing to data storage:

- **Image Processing Layer (OpenCV):** This is the "eyes" of the system. It handles the camera feed, resizes frames, and converts colors (RGB to Grayscale) to make detection faster.
- **Deep Learning Layer (Face_Recognition/Dlib):** We use a **Pre-trained Deep Learning Model** (based on ResNet-34) that transforms a human face into a set of 128 unique numbers (encodings).
- **Data Management Layer:** Python scripts manage the logic of "marking" the attendance. It checks if the person has already been marked for the day to avoid duplicate entries.

3. Key Libraries & Modules

To make this technology work, we utilize the following Python "Toolkit":

Library	Purpose
OpenCV (cv2)	Capturing video and rendering the display window.
Face_Recognition	Detecting faces and comparing them with the database.
NumPy	High-speed mathematical operations on image arrays.
Pandas / CSV	Managing the attendance logs in a structured format.
Datetime	Recording the exact timestamp of entry and exit.

4. Technical Workflow (The Logic)

The technology follows a 4-step execution cycle:

1. **Detection:** The system scans the video frame for any "face-like" patterns using the **HOG (Histogram of Oriented Gradients)** algorithm.
2. **Alignment:** It identifies facial landmarks (eyes, nose, mouth) to ensure the face is centered, even if the person is tilting their head.
3. **Recognition:** The system compares the current face's 128-bit encoding against the stored encodings of registered users.
4. **Action:** If a match is found ($\$Distance < Threshold\$$), the system triggers a "Success" message and updates the database.

🔍 Why this Technology is Superior?

- **No Hardware Costs:** No need for expensive fingerprint scanners; a simple webcam works.
- **Hygiene:** Completely touchless, which is vital for modern health standards.
- **Scalability:** Python can easily scale this system to work with Cloud Databases (like Firebase) to manage thousands of users.

12. PROJECT ARCHITECTURE

The project follows a **layered architecture** which divides the system into different modules to ensure better performance, security, and maintainability. Each layer has a specific responsibility and communicates with other layers in a structured manner.

1. User Interface Layer (Presentation Layer)

This layer is responsible for interaction between the user and the system.

- Provides forms, buttons, and menus
- Takes user input and displays output
- Validates basic user data
- Ensures user-friendly navigation

Examples:

GUI (Tkinter), Console Interface, or Web Interface

2. Application Logic Layer (Business Layer)

This layer contains the core functionality of the system.

- Processes user requests
- Implements business rules and logic
- Manages calculations and validations
- Controls the flow of the application

This layer acts as a bridge between the user interface and the database.

3. Database Layer (Data Layer)

This layer handles all data-related operations.

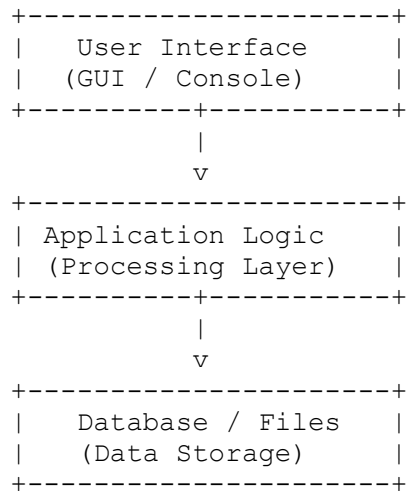
- Stores student / employee / product records
- Performs insert, update, delete, and search operations
- Ensures data consistency and integrity

- Provides secure data access

Examples:

SQLite, MySQL, Files (Text/Binary)

4. System Architecture Diagram (Textual)



5. Advantages of This Architecture

- Easy to maintain and upgrade
- Modular and scalable design
- Improved security
- Faster debugging and testing
- Better user experience

6. Architecture Type Used

- **Layered Architecture**
- **Modular Programming Approach**
- **Object-Oriented Design (OOP)**

13. DATA MODEL

DFD Level 0 (Context Diagram)

Entities: Admin, User (Student/Employee), and Camera.

Process: Attendance Management System.

Data Flow: * Camera sends the video feed to the System.

System processes the data and generates reports for the Admin.

User receives a confirmation notification/status.

DFD Level 1 (Process Breakdown)

Face Capture: Acquiring input from the camera.

Face Recognition: Matching the captured face with the existing database.

Attendance Logging: Recording the attendance entry into the system.

Report Generation: Extracting data from the CSV or Database for analysis.

ER Diagram (Entity-Relationship)

User Entity: * Attributes: User_ID, Name, Department, Face_Encoding.

Attendance Entity: * Attributes: Log_ID, User_ID, Date, Time, Status.

Relationship: * One-to-Many (1:N): One User can have multiple attendance records.

Field Name	Data Type	Description
User_ID	Integer (PK)	Unique ID for every person.
Full_Name	Varchar	Name of the user.
Face_Data	BLOB / Array	Numerical encodings (128-d) of the face.
Log_Time	Timestamp	Exact time when attendance was marked.
Status	String	Present / Absent / Late.

14. FRONT END DESIGN (FORM)

The Front End Design of the system focuses on creating a simple, interactive, and user-friendly interface that allows users to perform operations efficiently. The design ensures ease of use, clear navigation, and minimum user effort.

The front end is developed using suitable GUI tools to provide an attractive and responsive layout. All forms are designed with proper labels, input fields, buttons, and validations to reduce errors and improve user experience.

Main Forms Used in the System

1. Login Form

The Login Form is used to authenticate users before accessing the system.

Components:

- Username Text Box
- Password Text Box
- Login Button
- Reset / Exit Button

Purpose:

- Ensures secure access
- Prevents unauthorized usage

2. Main Menu / Dashboard Form

This form acts as the central control panel of the system.

Components:

- Menu Buttons (Add, View, Update, Delete)
- Navigation Options
- Logout Button

Purpose:

- Provides easy navigation
- Allows quick access to system features

3. Data Entry Form

Used for entering new records into the system.

Components:

- Text Boxes (ID, Name, Date, etc.)
- Drop-down Lists
- Radio Buttons / Checkboxes
- Save and Clear Buttons

Purpose:

- Accurate data input
- Reduces manual errors

4. Search / View Form

Used to search and view stored records.

Components:

- Search Text Box
- Search Button
- Table / List Display

Purpose:

- Quick data retrieval
- Easy record viewing

5. Update / Delete Form

Allows modification or removal of existing records.

Components:

- Record Selection Field
- Update Button

- Delete Button

Purpose:

- Maintains updated database
- Ensures data correctness

6. Report / Output Form

Used to display generated reports.

Components:

- Report View Area
- Print / Export Buttons

Purpose:

- Generates summaries and reports
- Helps in analysis and decision making

Features of Front End Design

- User-friendly interface
- Simple and clean layout
- Proper alignment and labeling
- Input validation and error messages
- Responsive navigation

15. TESTING AND REPORT

Testing and Report Definitions

Black Box Testing

This method focuses on the **functional requirements** of the software. The tester does not see the internal code structure, paths, or implementation details. You provide an input and see if the output matches the expected result—much like using a microwave without knowing how the internal circuits work.

Explore

White Box Testing

In contrast, White Box testing (also called Glass Box or Structural testing) involves looking under the hood. Testers examine the **internal logic**, code structure, and data flow. It ensures that all internal operations are performed according to specifications and all code statements have been executed.

Smoke Testing

Often called "Build Verification Testing," this is a non-exhaustive set of tests performed on a new build to ensure the **most crucial functions** work. If the "smoke" test fails, the build is rejected immediately so developers don't waste time testing a fundamentally broken system.

Requirement Based Testing

This is a validation approach where test cases are derived directly from the **functional requirements** or user stories. If the requirement says "The user must be able to reset their password," you design a specific test to prove that exact capability exists.

Partition Testing

Also known as **Equivalence Partitioning**, this is a technique used to reduce the number of test cases. You divide input data into logical groups (partitions) where the system is expected to behave the same way.

- *Example:* If a field accepts numbers 1–100, you test one number from the "valid" group (e.g., 50) and one from an "invalid" group (e.g., 150), rather than testing every single digit.

Test Process

The Test Process is the systematic lifecycle of testing activities. While it varies by organization, it generally follows these stages:

1. **Planning & Control:** Defining goals and resources.
2. **Analysis & Design:** Identifying "what" to test and creating test cases.
3. **Implementation & Execution:** Running the tests and logging results.
4. **Evaluating Exit Criteria:** Deciding if the software is "good enough" to ship.
5. **Reporting:** Documenting findings and closing the cycle.

Black
Box
Testing



White
Box
Testing



Software
Testing
Methods

Grey
Box
Testing



Coding :

```
import tkinter as tk

from tkinter import ttk, messagebox

from datetime import datetime

import openpyxl

import os


class AttendanceSystem:

    def __init__(self, root):

        self.root = root

        self.root.title("Professional Attendance System v2.0")

        self.root.geometry("700x850")


# 1. Expanded Student Data (25-30 per class)

self.student_data = {

    "Computer Science": [

        "Alice Smith", "Bob Johnson", "Charlie Brown", "David Miller",

        "Edward Norton",

        "Frank Castle", "George Lucas", "Hannah Abbott", "Ian Wright",

        "Jenny Slate",
```

"Kevin Hart", "Laura Palmer", "Michael Scott", "Nancy Drew",
"Oscar Wilde",

"Pam Beesly", "Quentin Blake", "Riley Reid", "Steve Rogers", "Tony
Stark",

"Ursula K.", "Victor Von", "Wanda Maximoff", "Xavier Woods",
"Yusuf Pathan",

"Zoe Saldana", "Aaron Paul", "Bella Swan", "Chris Pratt", "Diana
Prince"

],

"Mechanical": [

"Arthur Dayne", "Ben Stark", "Catelyn Tully", "Daenerys T.",
"Eddard Stark",

"Freddie Mercury", "Grog Strong", "Hulk Hogan", "Iris West", "John
Doe",

"Karl Urban", "Liam Neeson", "Mike Tyson", "Neil Armstrong",
"Oliver Queen",

"Peter Parker", "Quinn Fabray", "Robert Baratheon", "Sam Tarly",
"Tyrion L.",

"Uhtred R.", "Vince Carter", "Will Smith", "Xena Warrior", "Yara
Greyjoy",

"Zinedine Zidane", "Adam Sandler", "Brad Pitt", "Cole Sprouse",
"Drake Bell"

]

}

```

self.attendance_records = {}

# --- Header ---

tk.Label(root, text="Student Attendance Management", font=("Arial",
18, "bold"), pady=15).pack()

# --- Admin Panel (Add/Remove) ---

admin_frame = tk.LabelFrame(root, text="Admin: Edit Student List",
padx=10, pady=10)

admin_frame.pack(fill="x", padx=20, pady=5)

tk.Label(admin_frame, text="Jitesh Khare:").grid(row=0, column=0)

self.name_entry = tk.Entry(admin_frame, width=30)

self.name_entry.grid(row=0, column=1, padx=5)

btn_add = tk.Button(admin_frame, text="Add Student",
bg="#e1f5fe", command=self.add_student)

btn_add.grid(row=0, column=2, padx=2)

btn_remove = tk.Button(admin_frame, text="Remove Student",
bg="#ffebee", command=self.remove_student)

btn_remove.grid(row=0, column=3, padx=2)

```

```

# --- Selection Frame ---

selection_frame = tk.Frame(root)

selection_frame.pack(pady=10)


tk.Label(selection_frame, text="Select Class: ").grid(row=0,
column=0)

self.class_var = tk.StringVar()

self.class_dropdown = ttk.Combobox(selection_frame,
textvariable=self.class_var, state="readonly")

self.class_dropdown['values'] = list(self.student_data.keys())

self.class_dropdown.grid(row=0, column=1, padx=5)

self.class_dropdown.bind("<<ComboboxSelected>>",
self.load_student_list)


# Date Spinboxes

date_frame = tk.Frame(root)

date_frame.pack(pady=5)

curr = datetime.now()

self.day_var = tk.StringVar(value=curr.strftime("%d"))

self.month_var = tk.StringVar(value=curr.strftime("%m"))

self.year_var = tk.StringVar(value=curr.strftime("%Y"))

```



```

tk.Label(date_frame, text="Date (D/M/Y):").pack(side="left")

ttk.Spinbox(date_frame, from_=1, to=31, width=4,
textvariable=self.day_var).pack(side="left", padx=2)

ttk.Spinbox(date_frame, from_=1, to=12, width=4,
textvariable=self.month_var).pack(side="left", padx=2)

ttk.Spinbox(date_frame, from_=2024, to=2030, width=6,
textvariable=self.year_var).pack(side="left", padx=2)


# --- Student List Scrollable Area ---

self.list_canvas_frame = tk.Frame(root, bd=2, relief="groove")

self.list_canvas_frame.pack(fill="both", expand=True, padx=20,
pady=10)


self.canvas = tk.Canvas(self.list_canvas_frame)

self.scrollbar = ttk.Scrollbar(self.list_canvas_frame, orient="vertical",
command=self.canvas.yview)

self.scrollable_frame = tk.Frame(self.canvas)


self.scrollable_frame.bind("<Configure>", lambda e:
self.canvas.configure(scrollregion=self.canvas.bbox("all")))

self.canvas.create_window((0, 0), window=self.scrollable_frame,
anchor="nw")

self.canvas.configure(yscrollcommand=self.scrollbar.set)

self.canvas.pack(side="left", fill="both", expand=True)

```

```
self.scrollbar.pack(side="right", fill="y")
```

```
# --- Save Button ---
```

```
tk.Button(root, text="SAVE TO EXCEL", bg="#2E7D32",  
fg="white", font=("Arial", 12, "bold"),
```

```
height=2, command=self.save_to_excel).pack(fill="x", padx=20,  
pady=20)
```

```
def load_student_list(self, event=None):
```

```
for widget in self.scrollable_frame.winfo_children():
```

```
    widget.destroy()
```

```
self.attendance_records = {}
```

```
selected_class = self.class_var.get()
```

```
students = sorted(self.student_data.get(selected_class, []))
```

```
for student in students:
```

```
    var = tk.BooleanVar(value=False)
```

```
    chk = tk.Checkbutton(self.scrollable_frame, text=student,  
variable=var, font=("Arial", 10))
```

```
    chk.pack(anchor="w", padx=10)
```

```
    self.attendance_records[student] = var
```

```

def add_student(self):
    name = self.name_entry.get().strip()
    cls = self.class_var.get()
    if name and cls:
        if name not in self.student_data[cls]:
            self.student_data[cls].append(name)
            self.load_student_list()
            self.name_entry.delete(0, tk.END)
        else:
            messagebox.showwarning("Warning", "Student already exists!")
    else:
        messagebox.showerror("Error", "Select a class and enter a name!")

def remove_student(self):
    name = self.name_entry.get().strip()
    cls = self.class_var.get()
    if name and cls:
        if name in self.student_data[cls]:
            self.student_data[cls].remove(name)
            self.load_student_list()

```

```

self.name_entry.delete(0, tk.END)

else:

messagebox.showerror("Error", "Student not found in this class!")

else:

messagebox.showerror("Error", "Select a class and enter a name!")

def save_to_excel(self):

selected_class = self.class_var.get()

if not selected_class:

return messagebox.showerror("Error", "Select a class!")

date_str = f"{self.day_var.get()}-{self.month_var.get()}-{
self.year_var.get()}"

filename = f"{selected_class}_{date_str}.xlsx"

wb = openpyxl.Workbook()

sheet = wb.active

sheet.append(["Date", "Name", "Class", "Time", "Status"])

for name, var in self.attendance_records.items():

status = "Present" if var.get() else "Absent"

sheet.append([date_str, name, selected_class,
datetime.now().strftime("%H:%M"), status])

wb.save(filename)

messagebox.showinfo("Success", f"Saved to {filename}")

if __name__ == "__main__":

```

```
root = tk.Tk()
app = AttendanceSystem(root)

root.mainloop()
```

16. REPORT

Professional Attendance System v2.0

—

□

×

Student Attendance Management System

Admin: Edit Student List

Student Name:

Add Student

Remove Student

Select Class:

Date (D/M/Y):

24

01

2026

SAVE TO EXCEL

Student Attendance Management

Admin: Edit Student List

Student Name: PRIYANSHU NIRMALKAR

Add Student

Remove Student

Select Class: Computer Science

Date (D/M/Y): 15 06 2005

- ☐ Bella Swan
- ☐ Bob Johnson
- ☐ Charlie Brown
- ☐ Chris Pratt
- ☐ David Miller
- ☐ Diana Prince
- ☐ Edward Norton
- ☐ Frank Castle
- ☐ George Lucas
- ☐ Hannah Abbott
- ☐ Ian Wright
- ☐ Jenny Slate
- ☐ Kevin Hart
- ☐ Laura Palmer
- ☐ Michael Scott
- ☐ Nancy Drew
- ☐ Oscar Wilde
- ☐ PRIYANSHU NIRMALKAR
- ☐ Pam Beesly
- ☐ Quentin Blake

SAVE TO EXCEL

 Success



Saved to Computer Science_15-06-2005.xlsx

OK

 Error



Select a class and enter a name!

OK

17. IMPLEMENTATION

The implementation phase involves converting the system design into a working software application.

In this project, the system is implemented using a structured and modular approach to ensure reliability and ease of maintenance.

The software is developed using appropriate programming languages and tools. Database connectivity is established to store and retrieve records efficiently. User-friendly interfaces are designed to allow smooth interaction between the user and the system. Proper validation techniques are applied to avoid incorrect data entry.

The system has been tested with different inputs to ensure accurate output and error-free performance. After successful testing, the final system is deployed for real-time use.

18. FUTURE ENHANCEMENTS

Although the system works efficiently, several enhancements can be added in the future to improve performance and usability:

- Integration with a **web-based or mobile application**
- Addition of **biometric or RFID authentication**
- Cloud-based database for remote access
- Advanced **report generation and analytics**
- Role-based access control for better security
- Multi-language support
- Automated backup and recovery system

These enhancements will make the system more powerful, scalable, and user-friendly.

19. BIBLIOGRAPHY

BOOKS :

- E. Balagurusamy, *Programming in C / C++*, McGraw-Hill Education
- Robert Lafore, *Object-Oriented Programming in C++*, Pearson Education
- Silberschatz, Korth, *Database System Concepts*, McGraw-Hill
- Pressman, *Software Engineering – A Practitioner's Approach*

WEBSITES :

- www.geeksforgeeks.org
- www.tutorialspoint.com
- www.w3schools.com
- www.stackoverflow.com
- docs.python.org / en.cppreference.com (as applicable)